



파머완 9장 : 추천 시스템

01. 추천 시스템의 개요와 배경

- 추천 시스템(Recommendations) 을 통해 사용자의 취향을 이해하고 맞춤 상품과 콘텐츠를 제공해 조금이라도 오래 자기 사이트에 고객을 머무르게 하기 위해 전력을 기울임
- 온라인 스토어의 필수 요소



[추천 시스템의 유형]

- 콘텐츠 기반 필터링
- 협업 필터링
 - 최근접 이웃 협업 필터링
 - 잠재 요인 협업 필터링

02. 콘텐츠 기반 필터링 추천 시스템

- 콘텐츠 기반 필터링 방식 : 사용자가 특정한 아이템을 매우 선호하는 경우, 그 아이템과 비슷한 콘텐츠를 가진 다른 아이템을 추천하는 방식

ex) 사용자가 특정 영화에 높은 평점을 줌 → 그 영화의 장르, 출연 배우, 감독, 영화 키워드 등의 콘텐츠와 유사한 다른 영화를 추천해주는 방식



03. 최근접 이웃 협업 필터링

협업 필터링 : 사용자가 아이템에 매긴 평점 정보나 상품 구매 이력과 같은 사용자 행동 양식 (User Behavior)만을 기반으로 추천을 수행하는 것

협업 필터링의 주요 목표 : 사용자-아이템 평점 매트릭스와 같은 축적된 사용자 행동 데이터를 기반으로 사용자가 아직 평가하지 않은 아이템을 예측 평가(Predicted Rating)하는 것

- 최근접 이웃 방식과 잠재 요인 방식 모두 사용자-아이템 평점 행렬 데이터에만 의지해 추천
- 판다스의 pivot_table()과 같은 함수를 이용해 사용자-아이템 평점 행렬 형태로 변경해야함

로우 레벨 형태의 사용자 - 아이템 평점 데이터

UserID	Item ID	Rating
User 1	Item 1	3
User 1	Item 3	3
User 2	Item 1	4
User 2	Item 2	1
User 3	Item 4	5



사용자 로우, 아이템 칼럼으로 구성된 사용자 - 아이템 평점 데이터

	Item 1	Item 2	Item 3	Item 4
User 1	3		3	
User 2	4	1		
User 3				5

- 사용자 기반 : 당신과 비슷한 고객들이 다음 상품도 구매했습니다(Customers like you also bought these items).

→ 사용자 기반 최근접 이웃 방식은 특정 사용자와 유사한 다른 사용자를 Top-N으로 선정해 이 Top-N 사용자가 좋아하는 아이템을 추천하는 방식

		다크 나이트	인터스텔라	엣지오브 투모로우	프로메테우스	스타워즈 라스트제다이
상호간 유사도 높음	사용자 A	5	4	4		
	사용자 B	5	3	4	5	3
	사용자 C	4	3	3	2	5

사용자 A는 사용자 C보다 사용자 B와 영화 평점 측면에서 유사도가 높음. 따라서 사용자 A에게는 사용자 B가 재미있게 본 '프로메테우스'를 추천

- 아이템 기반 : 이 상품을 선택한 다른 고객들은 다음 상품도 구매했습니다(customers who bought this item also bought these items).
→ 아이템 기반 최근접 이웃 방식은 아이템이 가지는 속성과는 상관없이 사용자들이 그 아이템을 좋아하는지/싫어하는지의 평가 척도가 유사한 아이템을 추천하는 기준이 되는 알고리즘

		사용자 A	사용자 B	사용자 C	사용자 D	사용자 E
상호간 유사도 높음	다크 나이트	5	4	5	5	5
	프로메테우스	5	4	4		5
	스타워즈 라스트제다이	4	3	3		4

여러 사용자들의 평점을 기준으로 볼 때 '다크 나이트'와 가장 유사한 영화는 '프로메테우스'

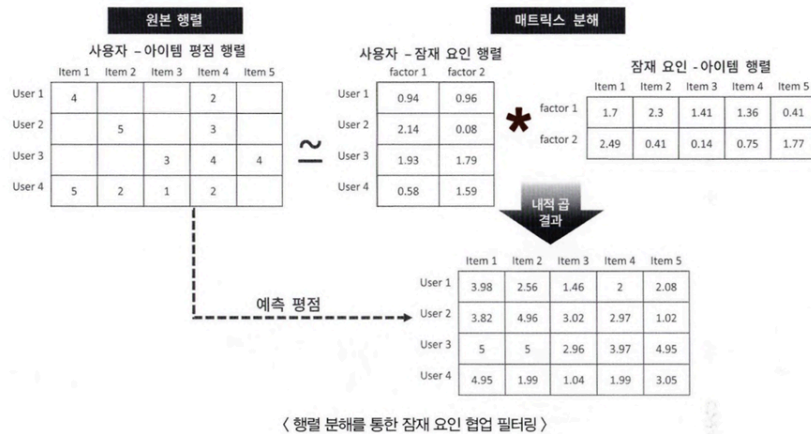
04. 잠재 요인 협업 필터링

잠재 요인 협업 필터링 : 사용자-아이템 평점 매트릭스

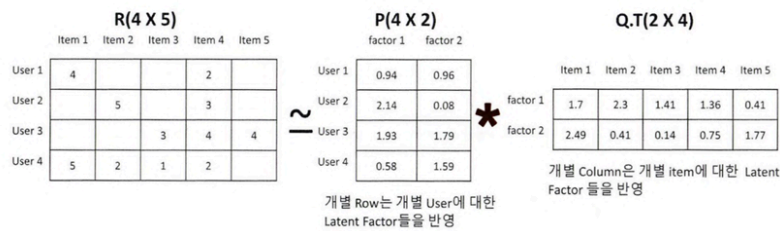
속에 숨어 있는 잠재 요인을 추출해 추천 예측을 할 수 있게 하는 기법

→ 행렬 분해 : 대규모 다차원 행렬을 SVD와 같은 차원 감소 기법으로 분해하는 과정에서 잠재 요인을 추출

잠재 요인을 기반으로 사용자-아이템 행렬 데이터를 저차원 밀집 행렬의 사용자-잠재 요인 행렬과 아이템-잠재 요인 행렬의 전치 행렬로 분해 ⇒ 두 행렬 내적을 통해 새로운 예측 사용자-아이템 평점 행렬 데이터를 생성



M개의 사용자(User) 행과 N개의 아이템(item)열을 가진 평점 행렬 R은 M X N 차원으로 구성되며, 행렬 분해를 통해서 사용자-K 차원 잠재 요인 행렬 P와 K 차원 잠재 요인 - 아이템 행렬 Q.T로 분해



$$r_{u,i} = p_u * q_i^t$$

$$R \cong \hat{R} = P * Q.T$$

확률적 경사하강법을 이용한 행렬 분해 : P와 Q 행렬로 계산된 예측 R 행렬 값이 실제 R 행렬 값과 가장 최소의 오류를 가질 수 있도록 반복적인 비용 함수 최적화를 통해 P와 Q를 유추

$$\min \sum (r_{u,i} - p_u q_i^t)^2 + \lambda (\|q_i\|^2 + \|p_u\|^2)$$

$$\begin{aligned} p'_u &= p_u + \eta (e_{(u,i)} * q_i - \lambda * p_u) \\ q'_i &= q_i + \eta (e_{(u,i)} * p_u - \lambda * q_i) \end{aligned}$$

08. 파이썬 추천 시스템 패키지 - Surprise

Surprise : 파이썬 기반의 추천 시스템 구축을 위한 전용 패키지 중의 하나

- Surprise 패키지 다운로드, ml-100k 데이터 세트 이용

Prediction 객체 : Surprise 패키지에서 제공하는 데이터 타입, Prediction 객체의 details 속성은 내부 처리 시 추천 예측을 할 수 없는 경우에 로

그용으로 데이터를 남기는 데 사용됨

```
algo = SVD(random_state=0)
algo.fit(trainset)
predictions = algo.test( testset )
print('prediction type :',type(predictions), ' size:',len(predictions))
print('prediction 결과의 최초 5개 추출')
predictions[:5]
```

```
[ (pred.uid, pred.iid, pred.est) for pred in predictions[:3] ]
```

```
uid = str(196)
iid = str(302)
pred = algo.predict(uid, iid)
print(pred)
```

```
accuracy.rmse(predictions)
```

결과

```
RMSE: 0.9467
0.9466860806937948
```

⇒ 추천 시스템 구현 완료!

Surprise 주요 모듈 소개

1. Dataset

: Surprise는 user_id(사용자 아이디), item_id(아이템 아이디), rating(평점) 데이터가 로우 레벨로 된 데이터 세트만 적용할 수 있습니다

Dataset.load_builtin (name='ml-100k')	무비렌즈 아카이브 FTP 서버에서 무비렌즈 데이터를 내려받음
Dataset.load_from_file (file_path, reader)	- os 파일에서 데이터를 로딩할 때 사용 - 콤마, 탭 등으로 칼럼이 분리된 포맷의 OS 파일에서 데이터를 로딩 - 입력 파라미터로 OS 파일명, Reader로 파일의 포맷을 지정
Dataset.load_from_df (df, reader)	- 판다스의 DataFrame에서 데이터를 로딩 - 파라미터로 DataFrame을 입력받으며 DataFrame 역시 반드시 3개의 칼럼 인 사용자 아이디, 아이템 아이디, 평점 순으로 칼럼 순서가 정해져 있어야 함 - 입력 파라미터로 DataFrame 객체, Reader로 파일의 포맷을 지정

2. OS 파일 데이터를 Surprise 데이터 세트로 로딩

ratings_noh.csv는 칼럼 헤더가 없고, 4개의 칼럼이 콤마로만 분리됨

Reader 클래스의 생성자에 각 필드의 칼럼명과 칼럼 분리문자, 그리고 최소~최대 평점을 입력해 객체를 생성하고, load_from_file()로 생성된 Reader 객체를 참조해 데이터 파일을 파싱하면서 로딩

```
trainset, testset = train_test_split(data, test_size=.25, random_state=0)
```

```
# 수행시마다 동일한 결과 도출을 위해 random_state 설정  
algo = SVD(n_factors=50, random_state=0)
```

```
# 학습 데이터 세트로 학습 후 테스트 데이터 세트로 평점 예측 후 RMSE 평가  
algo.fit(trainset)  
predictions = algo.test( testset )  
accuracy.rmse(predictions)
```

```
RMSE: 0.8682  
0.8681952927143516
```

3. 판다스 DataFrame에서 Surprise 데이터 세트로 로딩

Dataset.load_from_df()를 이용하면 판다스의 DataFrame에서도 Surprise 데이터 세트로 로딩할 수 있음.

→ DataFrame 역시 사용자 아이디, 아이템 아이디, 평점 칼럼 순서를 지켜야 함

```
import pandas as pd
from surprise import Reader, Dataset

ratings = pd.read_csv('/content/drive/My Drive/ratings.csv')
read = Reader(rating_scale = (0.5, 5.0))

#ratings DataFrame에서 칼럼은 사용자 아이디, 아이템 아이디, 평점 순서를 지켜야함
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)
trainset, testset = train_test_split(data, test_size = 0.25, random_state = 0)
algo = SVD(n_factors = 50, random_state = 0)
algo.fit(trainset)
predictions = algo.test(testset)
accuracy.rmse(predictions)
```

Surprise 추천 알고리즘 클래스

- SVD : 행렬 분해를 통한 잠재 요인 협업 필터링을 위한 SVD 알고리즘
- KNNBasic : 최근접 이웃 협업 필터링을 위한 KNN 알고리즘
- BaselineOnly : 사용자 Bias와 아이템 Biasfi 감안한 SGD 베이스라인 알고리즘

SVD 클래스 입력 파라미터

n_factors	잠재 요인 K의 개수
n_epochs	SGD 수행 시 반복 횟수
biased(bool)	베이스라인 사용자 편향 적용 여부(디폴트 = True)

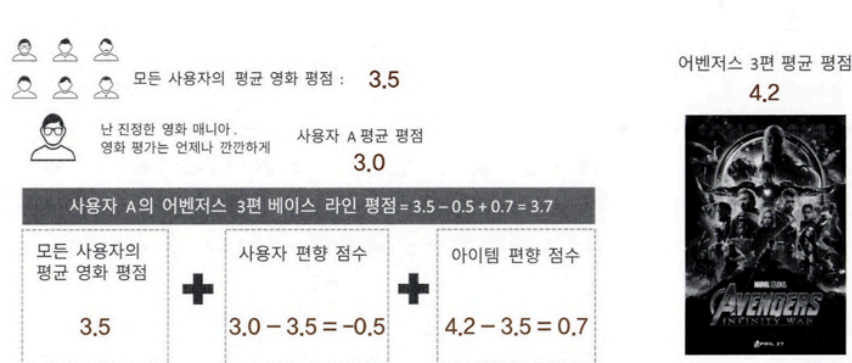
베이스라인 평점

: 개인의 성향을 반영해 아이템 평가에 편향성(bias) 요소를 반영하여 평점을 부과하는 것

⇒ 전체 평균 평점 + 사용자 편향 점수 + 아이템 편향 점수

- 전체 평균 평점 = 모든 사용자의 아이템에 대한 평점을 평균한 값
- 사용자 편향 점수 = 사용자별 아이템 평점 평균 값 - 전체 평균 평점

- 아이템 편향 점수 = 아이템별 평점 평균 값 - 전체 평균 평점



사용자 A의 '어벤저스 3'의 베이스라인 평점 : $3.5 - 0.5 + 0.7 = 3.7$

교차 검증과 하이퍼 파라미터 튜닝

- `cross_validate()`, `GridSearchCV`

```
from surprise.model_selection import cross_validate

# 판다스 DataFrame에서 Surprise 데이터 세트로 데이터 로딩
ratings = pd.read_csv('/content/drive/My Drive/ratings.csv')
reader = Reader(rating_scale=(0.5, 5.0))
data = Dataset.load_from_df(ratings[['userId', 'movieId', 'rating']], reader)

algo = SVD(random_state=0)
cross_validate(algo, data, measures=['RMSE', 'MAE'], cv=5, verbose=True)
```

`cross_validate()`는 위의 출력 결과와 같이 폴드별 성능 평가 수치와 전체 폴드의 평균 성능 평가 수치를 함께 보여줌

Surprise의 `GridSearchCV`도 사이킷런의 `GridSearchCV`와 유사하게 교차 검증을 통한 하이퍼 파라미터 최적화를 수행

```
def recomm_movie_by_surprise(algo, userId, unseen_movies, top_n=10):
    # 알고리즘 객체의 predict() 메서드를 평점이 없는 영화에 반복 수행한 후 결과를 list
    predictions = [algo.predict(str(userId), str(movieId)) for movieId in unseen_movies]

    def sortkey_est(pred):
        return pred.est
```

```

# sortkey_est( ) 반환값의 내림 차순으로 정렬 수행하고 top_n개의 최상위 값 추출.
predictions.sort(key=sortkey_est, reverse=True)
top_predictions= predictions[:top_n]

# top_n으로 추출된 영화의 정보 추출. 영화 아이디, 추천 예상 평점, 제목 추출
top_movie_ids = [ int(pred.iid) for pred in top_predictions]
top_movie_rating = [ pred.est for pred in top_predictions]
top_movie_titles = movies[movies.movielfid.isin(top_movie_ids)][ 'title']
top_movie_preds = [ (id, title, rating) for id, title, rating in zip(top_movie_ids,

return top_movie_preds

unseen_movies = get_unseen_surprise(ratings, movies, 9)
top_movie_preds = recomm_movie_by_surprise(algo, 9, unseen_movies, top_r
print('##### Top-10 추천 영화 리스트 #####')

for top_movie in top_movie_preds:
    print(top_movie[1], ":", top_movie[2])

```

```

평점 매긴 영화수: 46 추천대상 영화수: 9696 전체 영화수: 9742
##### Top-10 추천 영화 리스트 #####
Usual Suspects, The (1995) : 4.306302135700814
Star Wars: Episode IV - A New Hope (1977) : 4.281663842987387
Pulp Fiction (1994) : 4.278152632122759
Silence of the Lambs, The (1991) : 4.226073566460876
Godfather, The (1972) : 4.1918097904381995
Streetcar Named Desire, A (1951) : 4.154746591122657
Star Wars: Episode V - The Empire Strikes Back (1980) : 4.122016128534504
Star Wars: Episode VI - Return of the Jedi (1983) : 4.108009609093436
Goodfellas (1990) : 4.083464936588478
Glory (1989) : 4.07887165526957

```

⇒ 9번 아이디 사용자에게는 케빈 스페이시 주연의 '유주얼 서스펙트(Usual Suspect)', 그
리고 '펄프픽션', '양들의 침묵', '대부'와 '좋은 친구들' 같은 서스펜스/스릴러/범죄 영화 및 스
타워즈와 같은 액션 영화 등이 주로 추천됨

09. 정리

추천 시스템은 기업 애플리케이션에서 매우 중요한 위치를 차지함

keywords

- 콘텐츠 기반 필터링
- 협업 필터링 - 최근접 이웃 협업 필터링, 잠재 요인 협업 필터링
- Surprise 패키지